5. Constructors

★ Constructor –

A Constructor is a special method whose name is same as name of its class, which doesn't have any return type, which is called and executed by java compiler itself. when an instance of the class is created.

★ Important points about the Constructor –

1) A Constructor is a block of codes similar to the method. (or Constructor is a method of class).

2) Name of Constructor and name of its class are always same.

3) It is called when an instance of the class is created.

4) Constructor is a special type of method which is used to initialize the object.

5) Every time an object is created using the new() keyword, at least one constructor is called. It calls a default constructor if there is no constructor available in the class.

6) There are two types of Constructors in Java i.e. no-arg constructor and parameterized Constructor.

7) It is not necessary to write a Constructor for a

class. It is because java compiler creates a default constructor if your class doesn't have any.

8) It is called Constructor because it constructs the values at the time of object creation.

9) Constructor name must be the same as its class name.

10) A Constructor must have no explicit return type.

11) A java Constructor cannot be abstract, static, final and synchronized.

12) Note- We can use access modifiers while declaring a constructor. It controls the object creation. In other words, we can have private, protected, public or default constructor in Java.

13) Constructor returns the current class instance. (You cannot use return type yet it returns a value).

14) Constructor can perform other tasks instead of initialization like object creation, starting a thread, calling a method, etc. You can perform any operation in the Constructor as you perform in the method.

15) Java provides a Constructor class which can be used to get the internal information of a Constructor in the class. It is found in the java.lang.reflect package.

★ Types of Constructors-

In Java there are two types of Constructors -

I) Parameterized Constructor
II) Default Constructor (no-arg constructor)

I) Default Constructor -

A Constructor that doesnot have any parameters, is known as default constructor. The default constructor is used to provide the default values to the object like 0, null, etc, depending on the type. Such a default constructor is always automatically created by java compiler if there is not constructor in a class.

example-

```
Class addavg
{
  private int a, b, c;
  addavg()
  {
    a=20;
    b=30;
    c=40;
  }
  void display()
  {
    System.out.println("First number="+a);
    System.out.println("Second number="+b);
    System.out.println("Third number="+c);
  }
```

```
    Void result()
    {
        int add = a+b+c;
        double avg = (double) add/3;
        System.out.println("Addition = " + add);
        System.out.println("Average = " + avg);
    }
}
public class ConDemo
{
    public static void main(String args[])
    {
        System.out.println("Demo of Constructor");
        addavg obj = new addavg();
        obj.display();
        obj.result();
    }
}
```

II) Parameterized Constructor :-

" A Constructor which has a specific number of parameters is called as parameterized constructor.

The parameterized constructor is used to provide different values to distinct objects. However you can provide the Some values also.

example -

```
class addavg
{
  private int a,b,c;
  addavg ( int x, int y, int z)
  {
    a = x;
    b = y;
    c = z;
  }
  void display ()
  {
    System.out.println ("First number = " + a);
    System.out.println ("Second number = " + b);
    System.out.println ("Third number = " + c);
  }
  void result ()
  {
    int add = a+b+c;
    double avg = (double) add/3;
    System.out.println ("Addition = " + add);
    System.out.println ("Average = " + avg);
  }
}
class ConDemo
{
  public static void main ( String args[])
  {
    System.out.println ("Demo of Parameterized
                         Constructor");
    addavg obj = new addavg (40, 50, 60);
```

```
        obj.display();
        obj.result();
    }
}
```

☆ Constructor Overloading in Java -

A Constructor is just like method but without return type. It can also be overloaded like methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example -

```
class Student
{
  int id;
  String name;
  int age;
  Student (int i, String n)
  {
    id = i;
    name = n;
  }
  Student (int i, String n, int a)
  {
    id = i;
```

```
        name = n;
        age = a;
    }
    Void display ()
    {
        System.out.println (id + " " + name + " " + age);
    }
    public static void main (String args[])
    {
        Student S1 = new Student (111, "Ram");
        Student S2 = new Student (222, "Sham", 25);
        S1.display ();
        S2.display ();
    }
}
```

★ Copy Constructor –

There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another they are –

1) By Constructor.
2) By assigning the values of one object into another
3) By clone () method of Object class.

Examples –

1) By using Constructor –

```
class Student
{
    int id;
    String name;
    Student (int i, String n)
    {
        id = i;
        name = n;
    }
    Student ( Student s)
    {
        id = s.id;
        name = s.name;
    }
    void display ()
    {
        System.out.println ( id + " " + name);
    }
    public static void main ( String args [])
    {
        Student s1 = new Student (111, "Ram");
        Student s2 = new Student (s1);
        s1.display ();
        s2.display ();
    }
}
```

2) By assigning the values from one object to another

```
Class Student
{
    int id;
    String name;
    Student (int i, String n)
    {
        id = i;
        name = n;
    }
    Student ()
    {

    }
void display ()
    {
        System.out.println( id + "  " + name);
    }
    public static void main (String args[])
    {
        Student S1 = new Student (111, "Ram");
        Student S2 = new Student ();
        S2.id   = S1.id;
        S2.name = S1.name;
        S1.display ();
        S2.display ();
    }
}
```